03 | 11 | 25

2.3 INTERVALOS

En Haskell, los intervalos (o ranges) son una notación concisa para generar listas de valores que siguen una secuencia ordenada.

¿CÓMO FUNCIONAN?

Se escriben utilizando el operador de dos puntos (..) encerrados entre corchetes [...]. Para que un tipo de dato pueda usarse en un intervalo, debe pertenecer a la clase de tipos Enum (como números enteros, caracteres, etc.).

TIPOS INTER PRINCIPALES DE INTERVALOS

1. Intervalo básico (con inicio y fin): Genera una lista desde el primer valor hasta el último (incluyendo ambos, es decir inclusivo).

```
[1..10] -- [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
['a'..'e'] -- ['a', 'b', 'c', 'd', 'e']
```

2. Intervalo con paso (con saltos): Puedes definir los dos primeros elementos para indicarle a Haskell cuál es la diferencia o el "paso" entre cada número.

```
[1,3..10] -- [1, 3, 5, 7, 9] (salto de dos en dos)
[10,8..1] -- [10, 8, 6, 4, 2] (cuenta regresiva)
```

3. Intervalos infinitos (evaluación perezosa / lazy evaluation): Si omites el límite superior, Haskell crea una lista infinita. Gracias a la evaluación perezosa, la computadora no se congela a menos que intentes calcularla por completo; solo genera los elementos que tú le pidas

```
[1..] -- [1, 2, 3, 4, 5, 6, ...] (lista infinita de enteros)
take 5 [1..] -- [1, 2, 3, 4, 5] toma solo los primeros 5.
```

03 | 11 | 26

2.9 OPERADORES

En Haskell los operadores son funciones que se utilizan de forma infija (es decir, colocándose entre sus dos argumentos, como 1+2).

- Características principales

• Son funciones regulares: En Haskell, no hay una distinción real entre una función y un operador, cualquier función binaria puede usarse como operador y viceversa.

• Notación prefija e infija: Haskell utiliza las siguientes reglas sintácticas:

- De operador (infijo) a función (prefija): Si envuelves un operador entre paréntesis, puede usarse al principio como una función normal.

- De función (prefija) a operador (infijo): Si envuelve una función binaria entre comillos simples invertidas (backticks), puedes colocarla entre sus argumentos.

• Principales tipos de operadores en Haskell.

En el Preludio (Prelude) de Haskell vienen definidos por defecto los operadores más comunes:

1. Aritméticos: Suma (+), resta (-), multiplicación (*), división decimal (/), ^ y ^^ (exponenciación, además de funciones enteras como div, mod, rem y quot.
2. Relacionales y de comparación: == (igualdad), /= (desigualdad), <, <=, >, >=.
3. Lógicos (booleanos) && (AND lógico), || (OR lógico)
4. Listas: : (cons/agregar elemento al inicio de una lista), ++ (concatenar listas), !! (obtener el elemento en el índice n)
5. Composición y aplicación de funciones:
   1. . (composición matemática de funciones, e.g. f . g).
   2. $ (operador de aplicación que evita el exceso de paréntesis, e.g. f $ g x equivale a f (g x)).

2.7 EVALUACIÓN PEREZOSA

La evaluación perezosa (o lazy evaluation) es una estrategia en la que Haskell no calcula el valor de una expresión en el momento en el que se define, sino que retrasa el cálculo hasta que su resultado es estrictamente necesario para continuar el programa.

¿Cómo funciona?

- Bajo demanda: Si defines una variable o una función, Haskell solo crea una promesa (un thunk) de que calculará ese valor si alguien lo pide.
- Sin repetición: Una vez que se calcula por primera vez, el resultado se almacena (memorización) para que no sea necesario volver a procesarlo si se vuelve a usar.
- Comportamiento por defecto: En Haskell, prácticamente todas las expresiones son perezosas de manera predeterminada, a diferencia de lenguajes de evaluación estricta (voraz) como C++, Java o Python (por defecto)

VENTAJAS CLAVES EN EL DESARROLLO

1. Estructuras de datos infinitas: Puedes definir listas sin fin porque Haskell solo calculará los elementos que realmente le pidas.
2. Mayor eficiencia computacional: Evita procesar argumentos o ramas de código que nunca se utilizan en el resultado final.
3. Modularidad: Permite separar la definición de los datos (qué son) del flujo de control (cuántos necesito).